

Module 1: Introduction to Programming and C++

1.1 What is Programming

- Definition of a computer program: a set of instructions given to a computer to perform a specific task
- How a computer follows instructions step by step from top to bottom
- The role of a programmer: writing, testing, and fixing instructions
- Difference between hardware (physical parts) and software (programs and data)
- Types of programming languages
 - Machine language: binary code (0s and 1s) that the CPU understands directly
 - Assembly language: uses short human-readable codes like MOV, ADD that map to machine instructions
 - High-level languages: English-like syntax such as Python, Java, C++
 - Middle-level languages: combine high-level readability with low-level control, for example C and C++
- Compiled languages vs interpreted languages
 - Compiled: entire source code is translated to machine code before running (C, C++, Go)
 - Interpreted: code is translated and executed line by line at runtime (Python, JavaScript)
 - Compilation produces faster executable programs

1.2 Introduction to C++

- C++ was developed by Bjarne Stroustrup at Bell Labs starting in 1979
- Originally called "C with Classes" and renamed to C++ in 1983
- The ++ in the name comes from the increment operator in C, meaning an improved version of C
- Why C++ is widely used
 - Very fast execution speed due to direct memory control
 - Used in system software, game engines, embedded systems, and competitive programming
 - Large existing codebase and industry adoption
- C++ as a middle-level language
 - Provides high-level features like classes and objects
 - Also allows low-level operations like pointer arithmetic and direct memory management
- Relationship between C and C++
 - C++ is mostly backward compatible with C
 - Most valid C programs can be compiled as C++ programs
 - C++ adds object-oriented programming, templates, exceptions, and the standard library
- C++ standards over the years
 - C++98: the first standardized version

- C++11: introduced auto, nullptr, range-based for loop, lambda expressions
- C++14 and C++17: further improvements and additions
- Understanding which standard an IDE or compiler uses by default

1.3 Setting Up the Development Environment

- What a compiler is
- A compiler is a program that reads your source code (.cpp file) and translates it into machine code that the computer's processor can execute
- The translation happens all at once before the program runs, unlike an interpreter which runs code line by line
- The GNU C++ compiler (g++) is a free and widely used compiler included with MinGW on Windows
- During compilation, the compiler checks for syntax errors and reports them with line numbers before producing any output
- If there are no errors, the compiler produces an executable file (.exe on Windows) that can be run directly
- What an IDE is
- An IDE (Integrated Development Environment) is a software application that combines everything a programmer needs into one place
- It includes a code editor with syntax highlighting and auto-complete, a built-in compiler, a debugger, and a file manager
- Instead of using separate tools, you write, compile, and run code all from within the IDE
- Popular examples: Visual Studio Code, Code::Blocks, Dev-C++, CLion
- Setting up Visual Studio (recommended for beginners on Windows)
- Visual Studio is a full professional IDE made by Microsoft that comes with everything needed for C++ out of the box
- Download Visual Studio Community edition for free from visualstudio.microsoft.com (Community edition is completely free for students)
- Run the installer
- In the Visual Studio Installer, select the workload called "Desktop development with C++"
- Click Install; this automatically installs Visual Studio along with the Microsoft C++ compiler (MSVC), the debugger, and all required C++ tools
- No separate compiler download or PATH configuration is needed; everything is set up automatically
- Creating and running your first C++ program in Visual Studio
- Open Visual Studio and click "Create a new project"
- Select "Console App" under C++ and click Next
- Give the project a name, choose a save location, and click Create
- Visual Studio opens a ready-to-use main.cpp file with a basic Hello World program
- Write or modify the code in the editor

- Press F5 to compile and run the program with debugging; a console window opens showing the output
- Press Ctrl+F5 to run without the debugger (the console window stays open after the program finishes)
- What the C/C++ tools workload includes
- The MSVC compiler which compiles your .cpp files into executable programs
- IntelliSense: auto-complete suggestions and real-time error highlighting as you type
- A built-in debugger for setting breakpoints and inspecting variable values
- The Solution Explorer panel for managing all project files in one place
- Steps to write and run any C++ program
- Step 1: Write the source code in a .cpp file using a text editor or IDE
- Step 2: The compiler reads the source file and checks for errors
- Step 3: If no errors exist, the compiler produces an object file (.o) containing machine code
- Step 4: The linker combines the object file with any library code to produce the final executable (.exe on Windows)
- Step 5: Run the executable; the output appears in the terminal or console
- Types of errors a programmer encounters
- Syntax errors: violations of C++ grammar rules, caught by the compiler before the program runs, shown with line numbers in the error output
- Runtime errors: errors that occur while the program is running, such as dividing by zero or accessing memory that does not belong to the program
- Logic errors: the program compiles and runs without crashing but produces wrong or unexpected results due to a mistake in the programmer's reasoning

1.4 Structure of a C++ Program

- The #include directive
- Tells the preprocessor to include the contents of a header file
- #include <iostream> brings in the input and output functions
- The preprocessor runs before the compiler and handles directives starting with #
- The using namespace std statement
- Avoids writing std:: before every standard library function
- Without it, you must write std::cout and std::cin
- As programs grow larger, it is sometimes better to avoid this and use std:: explicitly
- The main() function
- Every C++ program must have exactly one main() function
- Execution always begins at the first line inside main()
- int before main means the function returns an integer value to the operating system
- Opening brace { and closing brace }
- Define the beginning and end of a block of code

- Every opening brace must have a matching closing brace
- Statements and semicolons
- Every statement in C++ ends with a semicolon
- A missing semicolon is one of the most common syntax errors
- The return 0 statement
- Signals to the operating system that the program ended successfully
- A non-zero return value typically indicates an error
- Case sensitivity
- C++ distinguishes between uppercase and lowercase letters
- Main is not the same as main
- cout is not the same as Cout
- Whitespace and indentation
- Spaces, tabs, and blank lines are ignored by the compiler
- Proper indentation makes code readable and easier to understand
- Consistent indentation is a professional habit

Example of a minimal C++ program:

```
#include <iostream>
using namespace std;
int main() {
    cout << "Hello, World!" << endl;
    return 0;
}
```